
Platformă de simulare autonomă a jocului de societate Catan cu agenți AI

Raport de proiect

Metode de Dezvoltare Software

Echipă: Rusu Iulia
Barcan Silviu-Ioan
Cocut Ioana

Disciplina: MDS, Anul 2, Semestrul 2

Facultatea: Matematică și Informatică,
Universitatea din București

Data: Septembrie 2026

Repository: <https://github.com/sigutz/Byte-Boards>

Producție: <https://www.bytenboards.games>

Cuprins

1	Introducere și context	3
1.1	Descrierea produsului	3
1.2	Stack tehnologic	3
1.3	Metodologia de dezvoltare	3
2	User Stories și Backlog	4
2.1	Epic 1 — Configurarea și urmărirea live a meciurilor	4
2.2	Epic 2 — Istoric și analitice	5
2.3	Epic 3 — Autentificare și proprietate	5
2.4	Epic 4 — Confidențialitate, partajare și administrare	6
2.5	Prioritizare prin metoda MoSCoW	6
3	Diagrame UML	7
3.1	Diagrama cazurilor de utilizare	7
3.2	Diagrama de clase (modelul de domeniu)	9
3.3	Diagrama de componente	9
3.4	Diagrama de desfășurare	10
3.5	Diagrama de secvență — ciclul de viață al unui meci	11
3.6	Diagrama de stare — statusul unui meci	13
3.7	Diagrama de activitate — algoritmul unei ture	13
3.7.1	Faza de producție	14
3.7.2	Faza de decizie a agenților	15
4	Source Control cu Git	17
4.1	Strategia de branching	17
4.2	Pull Requests	17
4.3	Tehnici Git utilizate	18
5	Teste automate și evaluări AI	18
5.1	Teste unitare pe motorul de joc	18
5.2	Testarea prin simulare de meci complet	19
5.3	Evals pentru agenți AI	19
6	Raportare de bug și rezolvare prin Pull Request	20
6.1	Bug #1 — Eșec de parsare JSON la autentificare	20
6.2	Bug #2 — Crash de runtime în modul Seafarers	20
6.3	Bug #3 — Cădere completă de producție: CI verde, imagine nefuncțională	20
6.3.1	Cauză rădăcină A — calea artefactului depinde de contextul de build	20
6.3.2	Cauză rădăcină B — servicii pe rețele Docker disjuncte	21
6.3.3	Remediere	21
6.3.4	Prevenirea recurenței	21
7	Pipeline CI/CD	23
7.1	Verificări automate	24
7.2	Gestionarea secretelor	24
7.3	Impactul pipeline-ului	24
8	Folosirea instrumentelor AI în dezvoltare	24
8.1	Claude Code — asistent de dezvoltare	25
8.2	Google Gemini — AI în produs	25

8.2.1	Sistemul de personalitate	25
8.2.2	Utilizări suplimentare	25
8.3	Lecții învățate	26
8.3.1	Ce a funcționat bine	26
8.3.2	Ce a necesitat judecată umană	26
8.3.3	Direcții viitoare	26
9	Concluzii	26

1 Introducere și context

1.1 Descrierea produsului

Proiectul implementează o platformă web în care agenți AI joacă autonom *Catan*, iar utilizatorii urmăresc partidele în timp real. Spre deosebire de o implementare clasică de joc, utilizatorul nu joacă — el configurează agenții, pornește meciul și observă deciziile luate de aceștia, însoțite de comentariu generat automat.

Sistemul suportă două seturi de reguli: **Catan Classic** (tablă de 19 hexagoane, 54 de noduri) și **Catan Seafarers** (extindere cu 3 insule exterioare, nave, pirat și puncte de victorie pentru explorare).

1.2 Stack tehnologic

Nivel	Tehnologie	Rol în sistem
Frontend	React 19 + Vite 7	SPA cu routing manual (<code>pushState</code>), fără librărie de router
Backend	Express 5 + Node.js	API REST (24 de rute) și flux SSE
ORM	Prisma 7 + <code>@prisma/adapters-pg</code>	9 modele; pool <code>pg</code> partajat pentru SQL brut
Bază de date	PostgreSQL	Persistență meciuri, agenți, evenimente
AI	Google Gemini	Decizii agenți, comentariu, rezumate, modulare
Infrastructură	Docker + Nginx	Build multi-stage, reverse proxy
CI/CD	GitHub Actions	6 job-uri de verificare + deploy prin SSH

Tabela 1: Stack-ul tehnologic al platformei

1.3 Metodologia de dezvoltare

Echipa a folosit un proces **iterativ-incremental** de tip Agile, cu livrare pe funcționalități:

- cerințele au fost formulate ca **user stories** cu criterii de acceptare explicite, grupate pe **epice**;
- prioritizarea backlog-ului s-a făcut prin metoda **MoSCoW**;
- fiecare funcționalitate a fost dezvoltată pe un **feature branch** izolat și integrată în **main** printr-un **Pull Request** cu review;
- integrarea a fost protejată de un **pipeline CI** care rulează teste, build și lint la fiecare push.

2 User Stories și Backlog

Backlog-ul a fost organizat pe 4 **epice**, cu 15 user stories. Fiecare poveste identifică explicit **actorul** (Vizitator, Utilizator autentificat, Administrator), ceea ce a permis derivarea directă a diagramei de cazuri de utilizare din Secțiunea 3.1.

2.1 Epic 1 — Configurarea și urmărirea live a meciurilor

US-01 — Urmărirea unui meci live

Ca Vizitator cu link către un meci, **vreau** să urmăresc agenții AI jucând în timp real, **pentru ca** să pot observa deciziile și strategiile lor.

Criterii de acceptare: Tabla se actualizează automat, fără refresh manual; fiecare mutare apare în maximum 3 secunde; fără link, un Vizitator nu poate descoperi meciuri.

US-02 — Alegerea jocului și a modului

Ca Utilizator autentificat, **vreau** să aleg între Catan Classic și Catan Seafarers, **pentru ca** să pot selecta setul de reguli dorit.

Criterii de acceptare: Selector vizibil la crearea meciului; în Seafarers sunt disponibile nave și puncte de victorie pentru insule.

US-03 — Pornirea unui meci la cerere

Ca Utilizator autentificat, **vreau** să pornesc un meci nou alegând agenții participanți, **pentru ca** să generez conținut nou de urmărit.

Criterii de acceptare: Minimum 2 agenți eligibili; limita de agenți este impusă în UI cu mesaj clar.

US-04 — Oprirea sau ștergerea unui meci

Ca Utilizator autentificat, **vreau** să opresc sau să șterg un meci pe care îl dețin, **pentru ca** să pot anula un meci blocat sau pornit din greșeală.

Criterii de acceptare: La „Stop” meciul se oprește și e marcat încheiat; la ștergere dispare din istoric.

US-05 — Comentariu live generat de AI

Ca Vizitator, **vreau** să văd comentariu generat de AI în timpul meciului, **pentru ca** experiența să fie mai captivantă.

Criterii de acceptare: La un eveniment semnificativ (comerț, hoț) apare o linie de comentariu în câteva secunde.

US-06 — Vizualizarea trăsăturilor agenților

Ca Vizitator, **vreau** să văd trăsăturile fiecărui agent, **pentru ca** să înțeleg *de ce* ia o anumită decizie, nu doar *ce* face.

Criterii de acceptare: Trăsăturile sunt afișate per agent; trăsăturile în conflict sunt rezolvate conform regulilor `TraitConflict` fără eroare.

2.2 Epic 2 — Istoric și analitice

US-07 — Navigarea în istoricul meciurilor

Ca Utilizator autentificat, **vreau** să răsfoiesc meciurile trecute, **pentru ca** să pot urmări tipare în timp.

Criterii de acceptare: Listă cu dată, tip de joc și agenți participanți.

US-08 — Detalii și statistici de meci

Ca Utilizator autentificat, **vreau** să deschid un meci trecut, **pentru ca** să analizez momentele cheie.

Criterii de acceptare: Log complet de evenimente per jucător, deținătorul drumului cel mai lung, scoruri finale.

US-09 — Filtrarea istoricului

Ca Utilizator autentificat, **vreau** să filtrez istoricul după tipul de joc, **pentru ca** să mă concentrez pe ce mă interesează.

Criterii de acceptare: La selectarea unui filtru sunt afișate doar meciurile de acel tip.

US-10 — Metrici de performanță per agent

Ca Utilizator autentificat, **vreau** să văd statistici agregate per agent, **pentru ca** să înțeleg care performează cel mai bine.

Criterii de acceptare: Rată de câștig, număr de meciuri, scor mediu, defalcate pe tip de joc.

2.3 Epic 3 — Autentificare și proprietate

US-11 — Înregistrare și autentificare

Ca Vizitator, **vreau** să îmi creez cont și să mă autentific, **pentru ca** să dețin agenți și să controlez cine îmi vede meciurile.

Criterii de acceptare: Parola stocată cu `bcrypt`; sesiune JWT; la credențiale greșite nu se creează sesiune.

US-12 — Deținerea agenților proprii

Ca Utilizator autentificat, **vreau** să dețin agenții pe care îi creez, **pentru ca** doar eu să îi pot configura sau înscrie în meciuri.

Criterii de acceptare: Agenții de sistem sunt vizibili tuturor; agenții proprii doar creatorului; numele sunt moderate automat înainte de salvare.

2.4 Epic 4 — Confidențialitate, partajare și administrare

US-13 — Controlul vizibilității meciurilor

Ca Utilizator autentificat, **vreau** să marchez meciurile ca private, protejate sau publice, **pentru ca** eu să decid cine le vede.

Criterii de acceptare: Un utilizator fără drept de acces nu poate vedea conținutul unui meci privat.

US-14 — Partajare prin link sau invitație

Ca Utilizator autentificat, **vreau** să generez un link de partajare bazat pe token, **pentru ca** să pot arăta un meci interesant unor persoane anume.

Criterii de acceptare: Cu tokenul valid, meciul poate fi vizualizat fără autentificare.

US-15 — Moderare de către administrator

Ca Administrator, **vreau** să pot șterge orice meci indiferent de proprietar, **pentru ca** să elimin conținut nepotrivit sau defect.

Criterii de acceptare: Rolul ADMIN are drept de ștergere pe orice resursă.

2.5 Prioritizare prin metoda MoSCoW

Prioritate	User Stories
Must have	US-01, US-02, US-03, US-05, US-07, US-08, US-11
Should have	US-04, US-09, US-10, US-12
Could have	US-06, US-13, US-14, US-15
Won't have (v1)	Multiplayer uman, chat live între utilizatori, turnee

Tabela 2: Prioritizarea cerințelor prin metoda MoSCoW

3 Diagrame UML

Modelarea sistemului folosește notația **UML 2.5**. Diagramele acoperă atât perspectiva **structurală** (cazuri de utilizare, clase, componente, desfășurare), cât și cea **comportamentală** (secvență, stare, activitate). Toate au fost realizate în **draw.io** și exportate vectorial (PDF), astfel încât să rămână clare la orice nivel de zoom.

3.1 Diagrama cazurilor de utilizare

Diagrama de cazuri de utilizare formalizează cerințele din Secțiunea 2. Sistemul are trei actori umani cu drepturi cumulative — **Vizitator** $\subset$ **Utilizator autentificat** $\subset$ **Administrator** — și un **actor secundar non-uman**, API-ul Gemini, invocat de sistem pentru deciziile agenților și generarea de text.

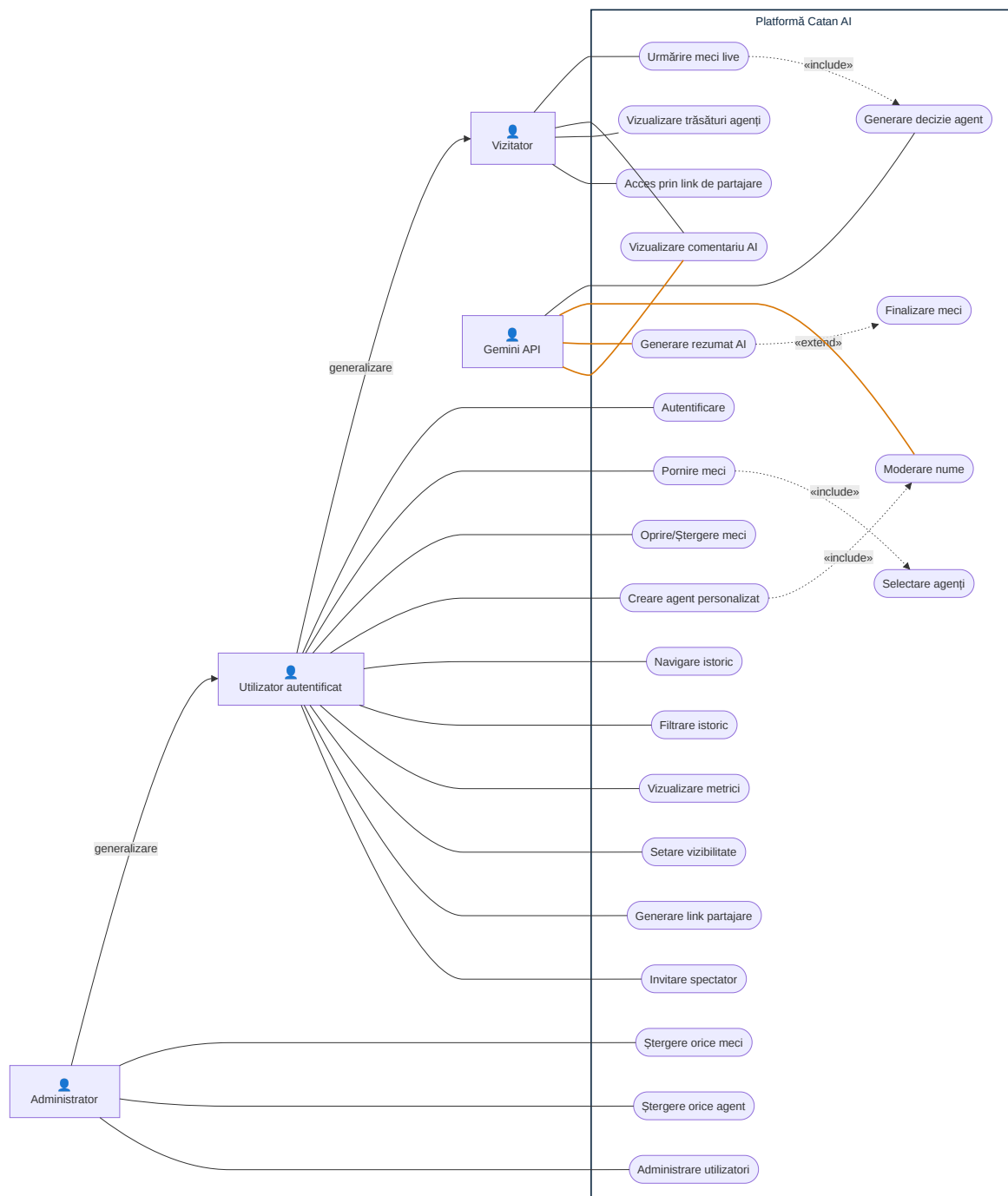


Figura 1: Diagrama cazurilor de utilizare — actori și funcționalități

Relațiile modelate sunt:

- **Generalizare** între actori: Administratorul moștenește tot ce poate face un Utilizator autentificat, care la rândul lui moștenește drepturile Vizitatorului.
- **«include»** — *Pornire meci* include obligatoriu *Selectare agenți* și *Moderare nume*.
- **«extend»** — *Generare rezumat AI* extinde *Finalizare meci*, fiind opțională (la indisponibilitatea API-ului se folosește un text implicit).

3.2 Diagrama de clase (modelul de domeniu)

Schema relațională este descrisă de 9 entități. Relația centrală este **MatchAgent** — o clasă de asociere între **Match** și **Agent**, care nu doar leagă cele două entități, ci poartă întreaga stare de joc a unui jucător într-un meci (resurse, clădiri, cărți de dezvoltare).

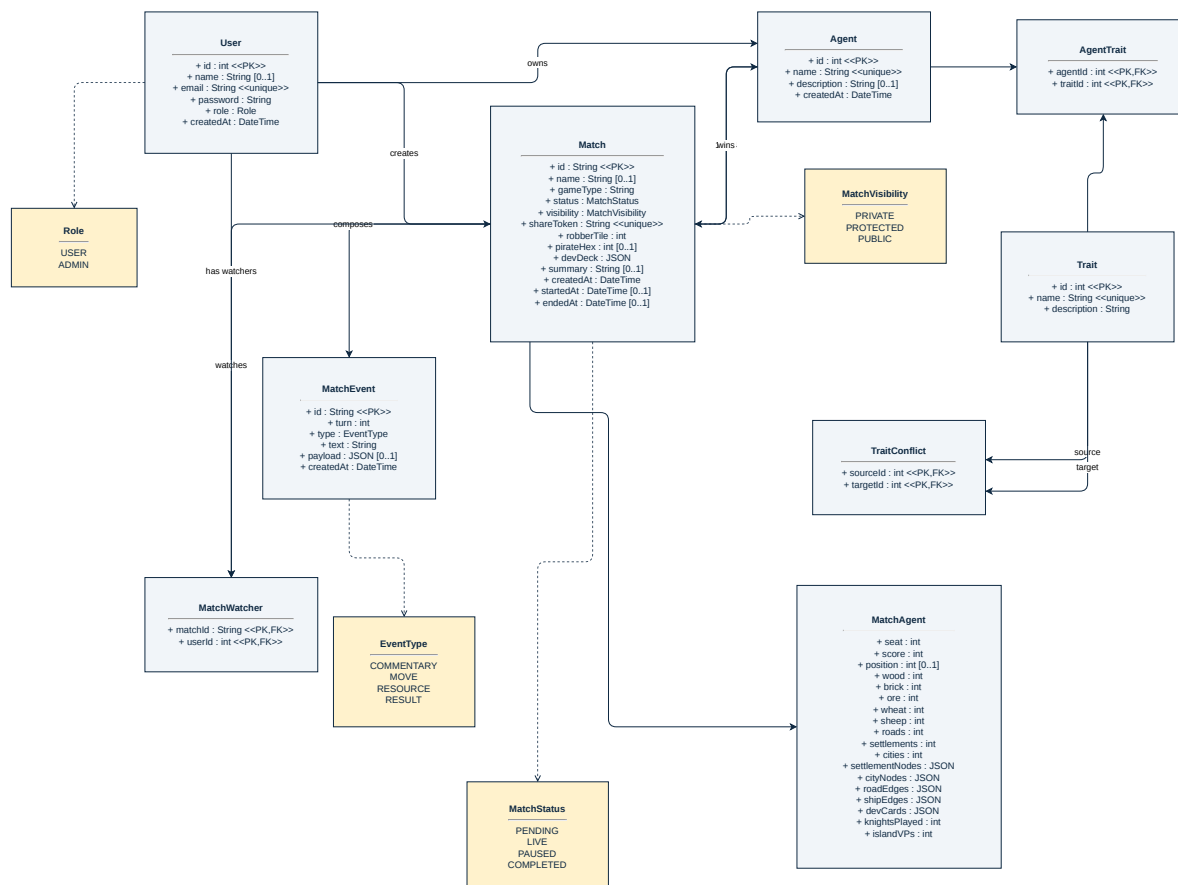


Figura 2: Diagrama de clase — modelul de domeniu cu multiplicități

Decizie de modelare

Trăsăturile agenților sunt modelate **relațional**, nu ca un câmp JSON: **Trait** (catalogul), **AgentTrait** (asocierea many-to-many) și **TraitConflict** (auto-asociere pe **Trait** care codifică perechile incompatibile). Astfel validarea combinațiilor invalide — de exemplu *Empathic* împreună cu *Aggressive* — se face prin interogare, nu prin logică hardcodată în aplicație.

3.3 Diagrama de componente

Diagrama de componente arată descompunerea logică a sistemului și **interfețele oferite și cerute** între module.

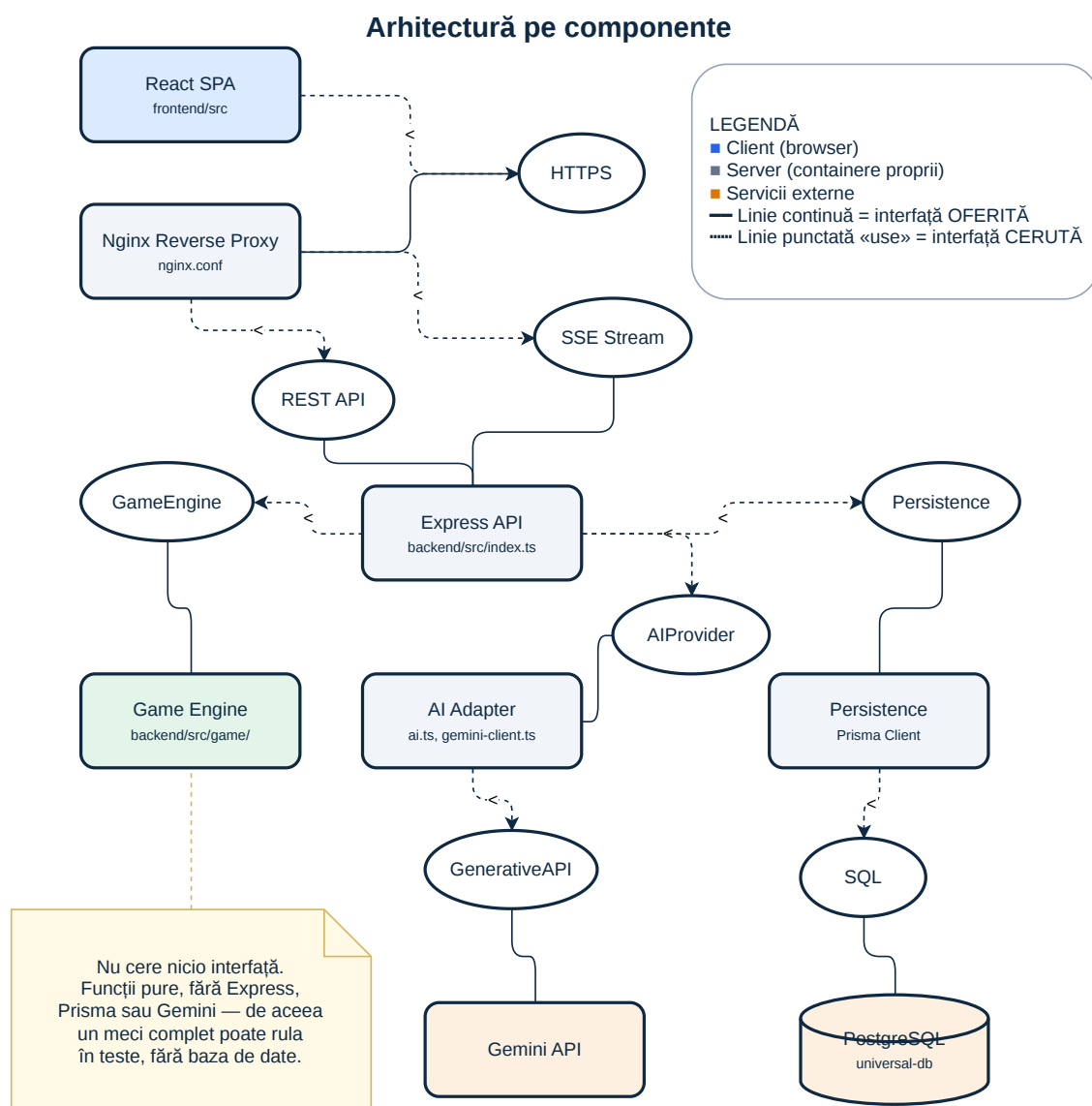


Figura 3: Diagrama de componente — module și interfețe

Motorul de joc este izolat complet de persistență: modulele din **backend/src/game/** sunt funcții pure, fără dependențe de Prisma sau Express. Această separare este ceea ce permite rularea unui meci complet în teste, fără bază de date (vezi Secțiunea 5).

3.4 Diagrama de desfășurare

Diagrama de desfășurare descrie topologia reală din producție: noduri fizice, containere Docker, rețele și protocoale.

Arhitectura Byte Boards

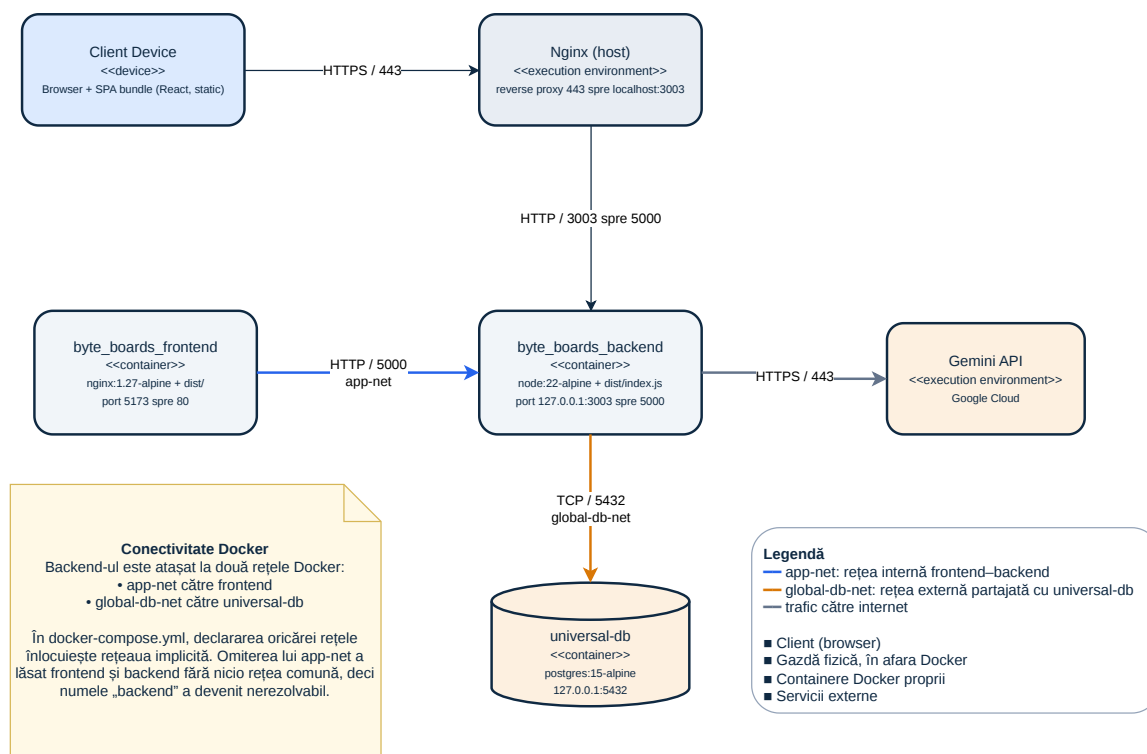


Figura 4: Diagrama de desfășurare — containere, rețele și porturi

Detaliu critic de infrastructură

Containerul **backend** este atașat la **două** rețele Docker: **app-net** (comunicarea cu frontend-ul) și **global-db-net** (rețea externă, partajată cu containerul de bază de date **universal-db**). Omiterea primei rețele a cauzat o cădere completă de producție, analizată în Secțiunea 6.3.

3.5 Diagrama de secvență — ciclul de viață al unui meci

Diagrama de secvență urmărește un meci de la cererea HTTP inițială până la persistarea rezumatului final, evidențiind natura **asincronă** a simulării: răspunsul HTTP se întoarce imediat, iar simularea continuă în fundal, comunicând cu clientul prin Server-Sent Events.

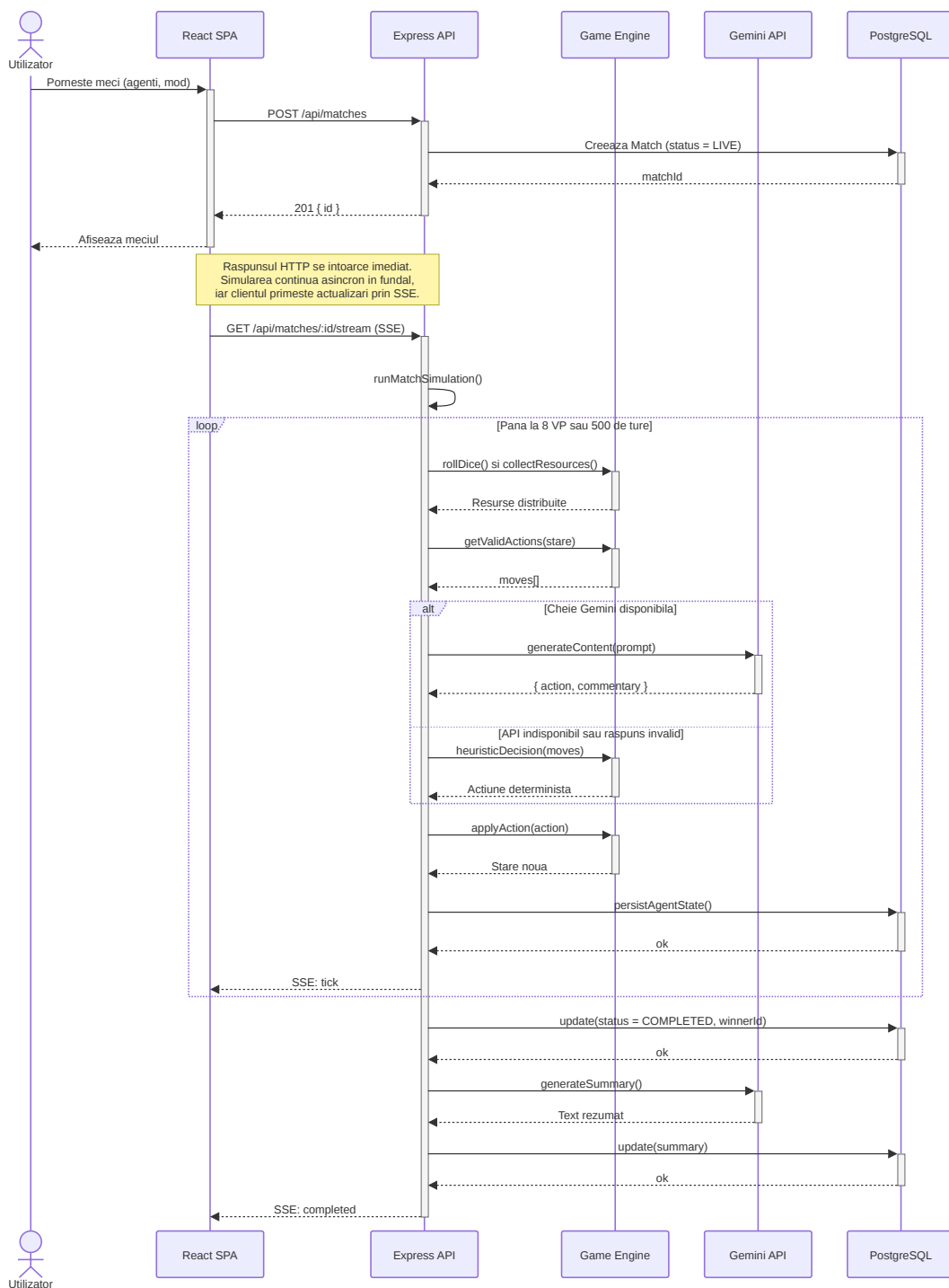


Figura 5: Diagrama de secvență — pornirea și derularea unui meci

Elementele importante sunt fragmentele combinate:

- **loop** — bucla de simulare, repetată până la atingerea a 8 puncte de victorie sau a limitei de 500 de ture;
- **alt** — ramificarea pe disponibilitatea API-ului Gemini: dacă apelul eșuează sau lipsește cheia, se aplică euristica deterministă;

- mesajul **asincron** (săgeată deschisă) pentru evenimentele SSE, spre deosebire de apelurile sincrone (săgeată plină).

3.6 Diagrama de stare — statusul unui meci

Entitatea `Match` are un ciclu de viață explicit, codificat în enumerarea `MatchStatus`.

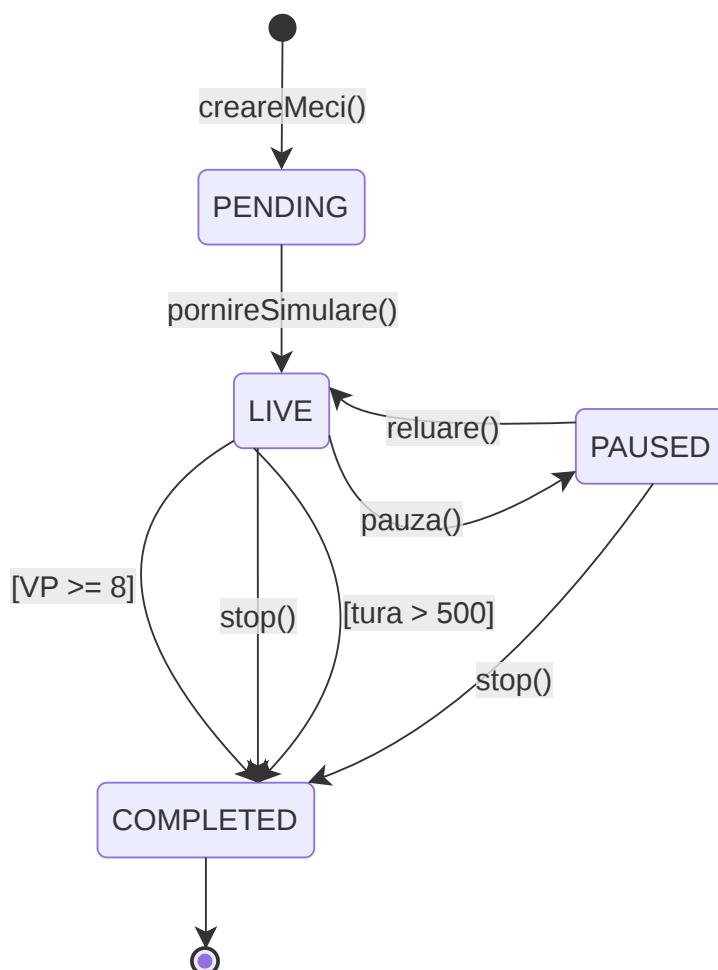


Figura 6: Diagrama de stare — tranzițiile unui meci

Tranziția `LIVE → PAUSED → LIVE` este reversibilă, în timp ce **COMPLETED** este o **stare finală**: odată atinsă, meciul nu mai poate fi repornit. Repornirea serverului declanșează tranziția de recuperare — meciurile rămase **LIVE** în baza de date sunt reluate automat din starea persistată.

3.7 Diagrama de activitate — algoritmul unei ture

O tură se desfășoară în două faze distincte: mai întâi **producția**, comună tuturor jucătorilor și determinată de zaruri, apoi **decizia**, repetată individual pentru fiecare agent. Cele două sunt prezentate separat, pentru lizibilitate.

3.7.1 Faza de producție

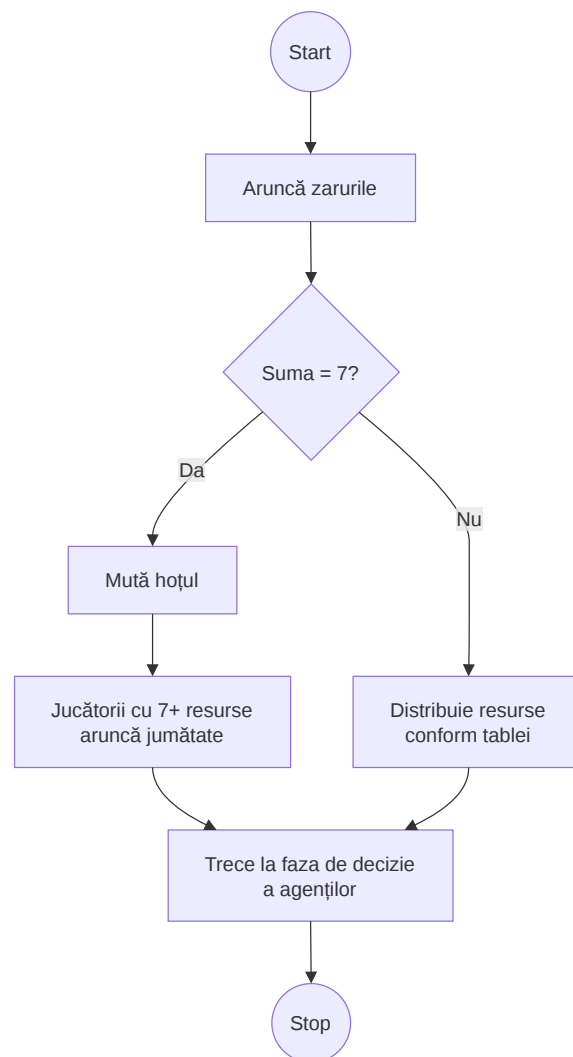


Figura 7: Diagrama de activitate — faza de producție a resurselor

Nodul de decizie pe rezultatul zarurilor ($= 7$ sau $\neq 7$) separă cele două regimuri de joc: producția normală de resurse, respectiv activarea hoțului cu descărcarea obligatorie a jucătorilor care dețin cel puțin 7 resurse. Această fază se execută o singură dată per tură, înaintea oricărei decizii de agent.

3.7.2 Faza de decizie a agenților

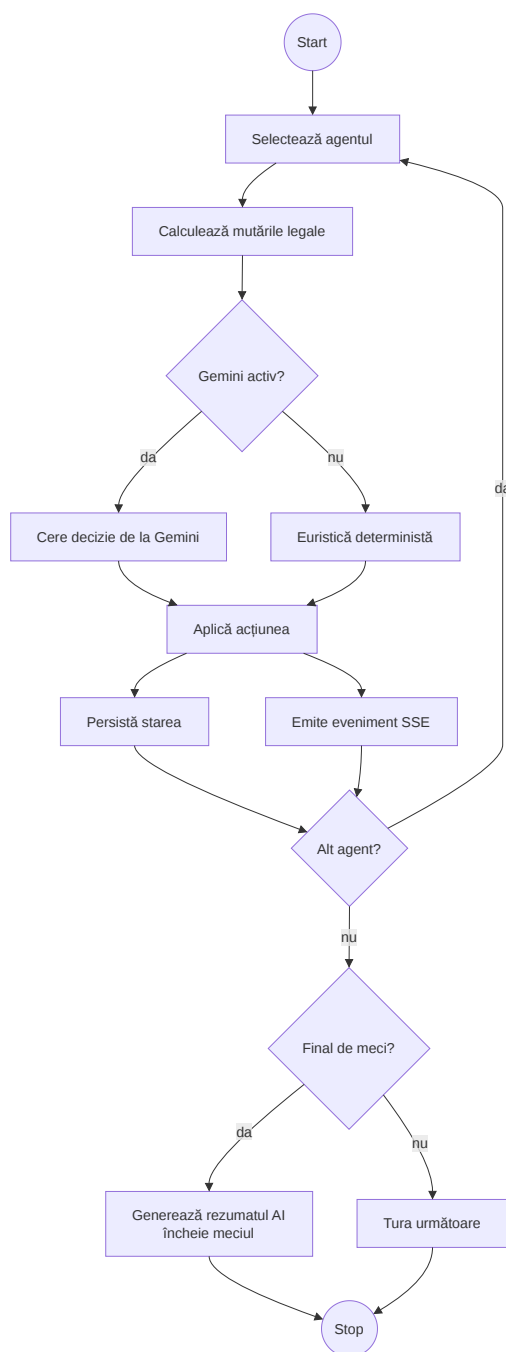


Figura 8: Diagrama de activitate — decizia și acțiunea unui agent

Bucula parcurge agenții în ordinea locurilor. Pentru fiecare, motorul calculează mai întâi mulțimea mutărilor legale; abia apoi intervine componenta AI, iar nodul de decizie pe disponibilitatea cheii Gemini arată explicit calea de rezervă: la eșec sau lipsă de cheie se aplică euristică deterministă, astfel încât simularea nu se blochează niciodată.

După aplicarea acțiunii, persistarea stării în baza de date și emiterea evenimentului SSE sunt **independente** între ele — niciuna nu așteaptă rezultatul celeilalte. Ieșirea din buclă se face fie la atingerea pragului de victorie, fie la depășirea limitei de ture.

4 Source Control cu Git

4.1 Strategia de branching

Proiectul folosește un model de tip **feature branching**: nicio modificare nu se face direct pe **main**, iar fiecare funcționalitate majoră este dezvoltată izolat și integrată printr-un Pull Request cu review.

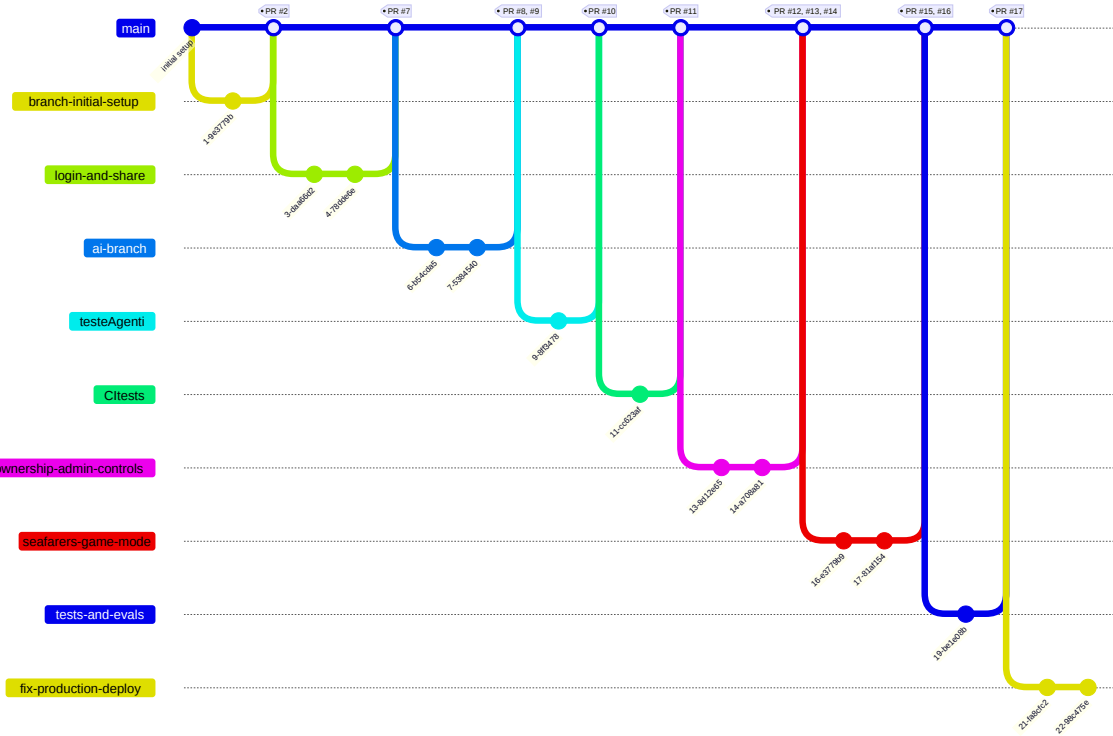


Figura 9: Strategia de branching — ramuri de funcționalitate și puncte de integrare

Branch	Scop	Autor
branch-initial-setup	Schelet inițial frontend + backend	Rusu Iulia
login-and-share	Autentificare JWT, share links, Docker	Rusu Iulia
ai-branch	Motor AI Gemini, graf tablă, personalitate	Barcan Silviu
testeAgenti	Suită de teste pentru logica jocului	Cocut Ioana
CItests	Workflow GitHub Actions	Cocut Ioana
bot-ownership-admin-controls	Proprietate agenți, rol admin, privacy	Rusu Iulia
seafarers-game-mode	Modul Seafarers complet cu AI agentiv	Rusu Iulia
tests-and-evals	Suită Vitest + evals DeepEval	Rusu Iulia
fix/production-deploy	Remediere cădere de producție	Barcan Silviu

Tabela 3: Ramurile create în cadrul proiectului

4.2 Pull Requests

Au fost integrate **13 Pull Requests**, toate cu review înainte de merge.

PR	Branch	Descriere
#2	branch-initial-setup	Setup inițial de proiect
#7	login-and-share	Autentificare, Docker, partajare
#8, #9	ai-branch	AI Gemini, graf tablă, personalitate
#10	testeAgenti	Teste automate pentru agenți
#11	CItests	Pipeline CI GitHub Actions
#12-#14	bot-ownership-admin-controls	Proprietate agenți + administrare
#15, #16	seafarers-game-mode	Modul Seafarers
#17	tests-and-evals	140 teste Vitest + 3 suite de evals

Tabela 4: Pull Requests integrate în main

4.3 Tehnici Git utilizate

- **Feature branches** — nicio modificare direct pe main;
- **Pull Requests cu code review** — toate integrările trec prin PR;
- **Rebase** — sincronizare cu main înainte de PR (`git pull --rebase origin main`);
- **Merge commits** — păstrate în istoric pentru trasabilitate;
- **Branch protection prin CI** — un PR nu poate fi integrat dacă vreo verificare automată eșuează.

5 Teste automate și evaluări AI

Testarea are **trei niveluri**, fiecare cu un scop distinct: teste unitare pe motorul de joc, teste de integrare pe API, și *evals* pentru componenta non-deterministă (LLM).

Nivel	Unealtă	Teste	Ce validează
Unitare	Vitest	140	Reguli de joc, graf tablă, trăsături, simulare completă
Integrare	Vitest + Supertest	(incluse)	Rute API, autorizare, degradare fără DB
Regresie	Node <code>assert</code>	15	Suita istorică <code>test-agents.ts</code>
Evals AI	DeepEval (Python)	3 suite	Calitatea deciziilor și textului generat de LLM

Tabela 5: Nivelurile de testare din proiect

5.1 Teste unitare pe motorul de joc

Modulele din `backend/src/game/` sunt funcții pure, deci pot fi testate fără bază de date sau server. Suita acoperă validarea acțiunilor legale (`getValidActions`), aplicarea efectelor (`applyAction`), calculul punctajului (`computeVP`), construcția grafului tablei și sistemul de trăsături.

```

1 cd backend && npm test
2
3 v src/game/personality.test.ts      (17 tests)
4 v src/game/graph.test.ts            (21 tests)
5 v src/game/seafarers-graph.test.ts  (16 tests)
6 v src/game/seafarers.test.ts        (18 tests)
7 v src/game/catan.test.ts            (49 tests)
8 v src/game/game-simulation.test.ts  (10 tests)
9 v src/api.test.ts                   ( 9 tests)
10

```

```

11 Test Files 7 passed (7)
12 Tests 140 passed (140)

```

Listing 1: Rularea suitei complete de teste

5.2 Testarea prin simulare de meci complet

Cel mai valoros test nu verifică o funcție, ci o **proprietate invariantă** pe parcursul unui meci întreg. Modulul `simulation-harness.ts` rulează un meci complet folosind exact aceleași funcții ca aplicația reală, dar fără persistență, iar testele verifică invarianți:

- există întotdeauna un câștigător (prin scor sau prin limita de ture);
- niciun jucător nu ajunge la o cantitate negativă de resurse;
- un nod nu poate fi simultan colonie și oraș;
- cel mult un jucător deține *Longest Road* la un moment dat, și doar cu un drum de lungime ≥ 5 ;
- cel mult un jucător deține *Largest Army*, și doar cu ≥ 3 cavaleri jucați.

Această abordare — *property-based testing* — prinde clase întregi de erori pe care testele punctuale nu le-ar detecta, pentru că explorează stări de joc generate aleator la fiecare rulare.

5.3 Evals pentru agenți AI

Componenta AI este **non-deterministă**: nu se poate scrie o aserțiune de egalitate pe ieșirea unui LLM. Soluția standard este evaluarea prin *metrici de calitate*, unde un al doilea model joacă rol de **judecător**. Proiectul folosește framework-ul **DeepEval** cu metrici de tip `GEval`, rulate ca teste `pytest` împotriva unor rute dedicate din backend.

Eval	Ce măsoară	Prag
TraitAlignment	Acțiunea aleasă este consistentă cu trăsăturile declarate ale agentului	0.6
CommentaryGrounding	Comentariul descrie fidel acțiunea, fără detalii inventate	0.6
Moderare nume	Acuratețea deciziilor de moderare pe 14 cazuri de test	$\geq 90\%$

Tabela 6: Metricile de evaluare a componentei AI

Garanție structurală înaintea evaluării calitative

Independent de calitatea răspunsului LLM, sistemul are o garanție **tare**: `getAgentDecision()` returnează întotdeauna o acțiune din lista de mutări legale. Dacă modelul propune o acțiune invalidă, dacă răspunsul nu e JSON valid, sau dacă apelul eșuează, se aplică euristică deterministă. **Nicio ieșire a modelului nu poate corupe starea jocului** — evals-urile măsoară *cât de bună* este decizia, nu *dacă este legală*.

6 Raportare de bug și rezolvare prin Pull Request

6.1 Bug #1 — Eșec de parsare JSON la autentificare

Simptom

După un deploy, pagina de login returna
JSON.parse: unexpected character at line 1 column 1 of the JSON data.

Investigație. Frontend-ul apela POST /api/auth/login și încerca să parseze răspunsul ca JSON, dar Nginx returna o pagină HTML de eroare 502 Bad Gateway. Cauza reală era că backend-ul nu pornise: procesul se oprea la **prisma db push**, pentru că schema conținea o coloană cu date existente pe care Prisma refuza să o șteargă fără confirmare explicită.

Cauză rădăcină. Dockerfile-ul de producție nu includea flagul `--accept-data-loss`. **Rezolvare:** PR #15, cu reproducere locală și review.

6.2 Bug #2 — Crash de runtime în modul Seafarers

Simptom

Meciurile Seafarers se opreau după câteva ture cu
TypeError: Cannot read properties of undefined (reading 'adjacentNodes').

Investigație. Funcția `getValidActions` caută nodurile adiacente în structura `BOARD_NODES`, definită pentru tabla clasică (nodurile 0–53). În Seafarers, jucătorii pot coloniza nodurile exterioare 54–71, care nu existau în acea structură; accesarea returna `undefined`.

```
1 // ÎNAINTE:
2 for (const adjNode of BOARD_NODES[nodeId].adjacentNodes) {
3
4 // DUPĂ:
5 for (const adjNode of (BOARD_NODES[nodeId]?.adjacentNodes ?? [])) {
```

Listing 2: Fix aplicat — optional chaining pentru noduri exterioare

Nodurile exterioare sunt astfel ignorate la calculul drumurilor — ceea ce este corect din punct de vedere al regulilor, drumurile neputând fi construite pe mare. **Rezolvare:** PR #15.

6.3 Bug #3 — Cădere completă de producție: CI verde, imagine nefuncțională

Acesta este cel mai instructiv defect al proiectului, pentru că ilustrează o categorie de erori pe care testele unitare **nu o pot prinde prin construcție**: o divergență între mediul de test și artefactul livrat.

Simptom

Ambele containere în restart infinit. Backend: Cannot find module
'/app/dist/src/index.js'. Frontend: [emerg] host not found in upstream
"backend".

6.3.1 Cauză rădăcină A — calea artefactului depinde de contextul de build

Fișierul `tsconfig.json` nu declara explicit `rootDir`. În lipsa lui, TypeScript îl **deduce** din prefixul comun al fișierelor de intrare — deci calea de ieșire depinde de *ce fișiere sunt prezente*:

Context	Fișiere de intrare	Ieșire tsc
Checkout complet (CI, local)	src/**/* + test-agents.ts	dist/src/index.js
Imagine Docker	doar src/**/*	dist/index.js

Tabela 7: Divergența căii de ieșire între CI și imaginea Docker

Dockerfile-ul copiază doar directorul `src/`, nu și `test-agents.ts`. Un commit anterior eliminase `rootDir`; un commit ulterior a „corectat” comanda de pornire către `dist/src/index.js`, potrivit-o cu ce producea CI-ul — dar nu cu ce producea imaginea. **Rezultat: CI verde, imagine care nu putea porni niciodată.**

6.3.2 Cauză rădăcină B — servicii pe rețele Docker disjuncte

În `docker-compose.yml`, serviciul `backend` declara doar rețeaua `global-db-net`. În Docker Compose, **declararea oricărei rețele înlocuiește rețeaua implicită**, deci `frontend` și `backend` nu mai aveau nicio rețea comună, iar numele `backend` devenea nerezolvabil. Nginx rezolvă gazda dintr-un `proxy_pass` literal **o singură dată, la pornire**, și refuză să pornească dacă rezoluția eșuează — astfel cădea și `frontend`-ul.

6.3.3 Remediere

```
1 // tsconfig.json -- rootDir fixat explicit
2 "rootDir": "./src", // iesirea nu mai depinde de contextul de build
3 "include": ["src/**/*"],
4 "exclude": ["node_modules", "dist", "src/**/*test.ts"]
```

Listing 3: Fixarea căii de ieșire a compilatorului

```
1 RUN npm run build
2 RUN test -f dist/index.js
```

Listing 4: Verificare la build — imaginea eșuează în loc să pornească defect

Pentru rețea s-a introdus o rețea comună `app-net`, iar configurația Nginx a fost transformată în *template* randat de entypoint-ul imaginii, cu rezolvare **leneșă** a upstream-ului printr-o variabilă. Efect: Nginx pornește chiar și fără `backend` disponibil, degradează la 502 și **își revine automat** când `backend`-ul redevine disponibil, fără restart.

6.3.4 Prevenirea recurenței

Golul real era că CI valida `npm run build` pe un checkout complet — ceea ce **nu** este ce face imaginea. S-a adăugat un job care construiește imaginile reale, verifică existența punctului de intrare și validează configurația Nginx fără upstream disponibil:

```
1 - run: docker build -t bytenboard-backend ./backend
2 - run: docker run --rm bytenboard-backend test -f dist/index.js
3 - run: docker build --build-arg VITE_API_BASE_URL= -t bytenboard-frontend ./frontend
4 - run: docker run --rm bytenboard-frontend nginx -t
```

Listing 5: Job-ul CI care previne recurența

Lecție de proces

Un pipeline CI nu validează codul, ci **artefactul pe care îl livrează**. Dacă mediul de test diferă de mediul de producție — fie și doar prin lista de fișiere copiate — testele pot trece

în timp ce produsul livrat este nefuncțional.

7 Pipeline CI/CD

Proiectul are două workflow-uri GitHub Actions distincte: `test.yml` (rulează la orice push sau PR, pe orice ramură) și `deploy.yml` (rulează doar la push pe `main` și livrează în producție).

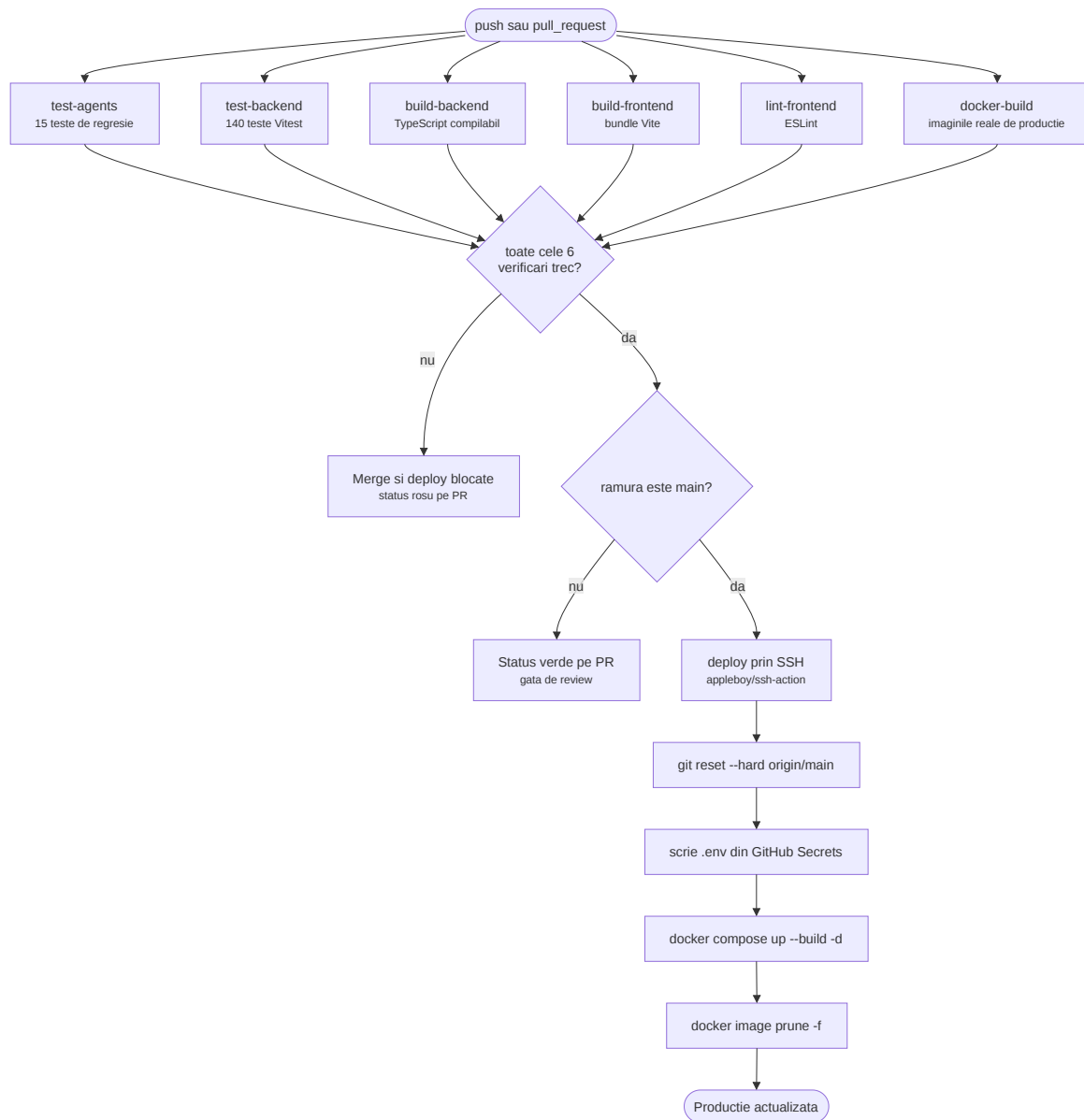


Figura 10: Pipeline-ul CI/CD — verificări automate și livrare

7.1 Verificări automate

Job	Ce verifică	Eșuează dacă
test-agents	Suita istorică de regresie	O aserțiune eșuează
test-backend	140 de teste Vitest	Orice test pică
build-backend	TypeScript compilabil	Erori de tip sau import
build-frontend	Bundle Vite compilabil	Erori TS, importuri lipsă
lint-frontend	ESLint	Variabile neutilizate, tipuri any
docker-build	Imaginile reale de producție	Punctul de intrare lipsește sau Nginx e invalid
deploy	Livrare prin SSH	Oricare dintre cele de mai sus eșuează

Tabela 8: Verificările automate din pipeline

Ultimele două job-uri (`test-backend`, `docker-build`) au fost adăugate ca urmare directă a incidentului din Secțiunea 6.3.

7.2 Gestionarea secretelor

Pipeline-ul injectează secretele din GitHub Secrets într-un fișier `.env` generat pe server la fiecare deploy:

- `DATABASE_URL`, `JWT_SECRET`, `VITE_API_BASE_URL`;
- `GEMINI_1 ...GEMINI_8` — pool-ul de 8 chei API;
- `SERVER_HOST`, `SERVER_USER`, `SERVER_SSH_KEY`.

Niciun secret nu este versionat: `.env` figurează în `.gitignore`.

7.3 Impactul pipeline-ului

Politica „niciun merge fără verificări verzi” a prins activ:

- erori TypeScript introduse de câmpuri noi (`shipEdges`, `islandVPs`) neadăugate în factory-ul de test;
- erori ESLint preexistente, expuse de verificarea adăugată în PR #12;
- blocaje de deploy cauzate de un build de frontend eșuat.

8 Folosirea instrumentelor AI în dezvoltare

Proiectul implică AI pe **două axe distincte**, care nu trebuie confundate: AI ca *instrument de dezvoltare* și AI ca *funcționalitate a produsului*.

Instrument	Rol	Context de utilizare
Claude Code (Anthropic)	Asistent de programare interactiv	În terminal, pe parcursul dezvoltării
Google Gemini API	Motorul de decizie al agenților	Integrat în produs ca funcționalitate centrală
DeepEval + Gemini	Model-judecător pentru evals	În suita de testare a componentei AI

Tabela 9: Instrumentele AI utilizate în proiect

8.1 Claude Code — asistent de dezvoltare

Claude Code a fost folosit ca *pair-programmer* în terminal, cu acces la fișierele proiectului. Spre deosebire de autocomplete, operează la nivel de **task**, nu de linie: primește o descriere în limbaj natural, citește fișierele relevante, raționează arhitectural și aplică modificări coordonate pe mai multe fișiere.

Arii susținute:

- **Arhitectură backend** — reprezentarea grafică a tablei (`graph.ts`, `seafarers-graph.ts`), sistemul de trăsături cu matrice de conflicte, și extensia Seafarers fără a rupe logica Classic. Decizia cheie — `SeafarersContext` ca parametru *optional* — a permis zero cuplare între moduri.
- **Frontend** — rescrierea `CatanBoard.tsx` pentru Seafarers (canvas extins, hexagoane exterioare, trasee punctate pentru nave).
- **DevOps** — build-uri Docker multi-stage, pipeline CI/CD, și diagnoza incidentului din Secțiunea 6.3, unde cauza rădăcină a fost **demonstrată experimental** prin reproducerea configurației vechi în contextul real de build Docker.
- **Testare** — generarea suitei Vitest și a harness-ului de simulare.

8.2 Google Gemini — AI în produs

Gemini este apelat la fiecare tură a fiecărui agent prin `backend/src/game/ai.ts`, gestionat de `gemini-client.ts` cu un pool de 8 chei API în rotație. La eroare recuperabilă (429, 503, timeout), clientul rotește automat cheia.

8.2.1 Sistemul de personalitate

Fiecare agent are trăsături alese din **15 posibile**: *Empathic*, *Greedy*, *HardTrader*, *Aggressive*, *Expansionist*, *Defensive*, *SmartBuilder*, *FastSpender*, *DevFocused*, *Diplomatic*, *Opportunist*, *Militarist*, *Settler*, *Merchant*, *Tactician*. Trăsăturile sunt stocate relațional și injectate în system prompt, producând comportamente strategice distincte. Perechile incompatibile sunt respinse la creare, pe baza tabeli `TraitConflict`.

8.2.2 Utilizări suplimentare

- **Moderare de conținut** — numele agenților și meciurilor sunt validate înainte de stocare, cu o listă locală de termeni interziși ca primă barieră (funcționează și când API-ul este indisponibil);
- **Rezumate narrative** — la finalul fiecărui meci se generează un rezumat stocat în `Match.summary`;
- **Comentariu live** — o frază per acțiune, difuzată prin SSE.

Estimare de consum: $\approx 40\text{--}80$ de apeluri per meci pentru decizii, plus unul pentru rezumatul final. Fallback-ul determinist garantează că jocul funcționează și fără nicio cheie configurată.

8.3 Lecții învățate

8.3.1 Ce a funcționat bine

- utilizarea asistentului la nivel de task (nu de linie) — semnificativ mai productivă pentru modificări cross-file;
- parametrul opțional `SeafarersContext` — extindere fără regresie pe modul Classic;
- fallback-ul determinist — dezvoltare și testare fără consum de cotă API;
- **verificarea empirică a ipotezelor** — în incidentul de producție, cauza rădăcină nu a fost presupusă, ci reprodusă și demonstrată.

8.3.2 Ce a necesitat judecată umană

- decizii de schemă (când `--accept-data-loss` este acceptabil față de o migrare propriu-zisă);
- granițele arhitecturale între modurile de joc;
- compromisuri cost/calitate în verbozitatea prompt-urilor.

8.3.3 Direcții viitoare

- trecerea de la `prisma db push` la migrări versionate (`prisma migrate deploy`);
- paginarea log-ului de evenimente și transmiterea incrementală prin SSE;
- caching pentru stări de joc repetitive, pentru reducerea costului API;
- instrumentarea latenței deciziilor AI.

9 Concluzii

Proiectul a livrat o platformă funcțională, desfășurată în producție, care acoperă toate cele 15 user stories din backlog. Din perspectiva *Metodelor de Dezvoltare Software*, elementele cele mai relevante sunt:

- **Proces** — backlog structurat pe epice, prioritizare MoSCoW, dezvoltare pe feature branches, integrare exclusiv prin Pull Requests cu review;
- **Modelare** — sistemul a fost descris prin șapte diagrame UML complementare, acoperind atât structura, cât și comportamentul;
- **Calitate** — trei niveluri de testare, inclusiv testare bazată pe proprietăți pentru simulare și *evals* pentru componenta non-deterministă;
- **Automatizare** — pipeline CI/CD cu șase verificări obligatorii înaintea livrării automate în producție;
- **Învățare din incidente** — căderea de producție analizată în Secțiunea 6.3 a produs o îmbunătățire permanentă de proces, nu doar o remediere punctuală.

Cea mai importantă lecție tehnică a proiectului este că **o suită de teste verde nu garantează un produs funcțional** dacă mediul validat diferă de artefactul livrat — observație care a schimbat structura pipeline-ului.